

Exit status and conditional chaining

Every command leaves an exit status

Zero, non-zero, and short-circuit

- Exit zero is the success convention Make, CI, and other scripts rely on
- Non-zero signals an error, often one for a generic fail, two for bad usage
- Dollar-question-mark is the last command's code
- True and false are tiny commands that exit zero or one so you can practice chaining
- Prefer if for real branches

Browser lab

Digital Design and Verification Platform

HomeToolsExit status

Watch `$?` after each command, and see how `&&` / `||` short-circuit — including the classic chain pitfall.

Starter example: `Run true; echo $? (0) then false; echo $? (1). Compare true && echo ok with false && echo ok, and try false || echo fallback.`

Load starter example

Challenges 0 / 22 cleared

Quiz: success: Exit status for success is? Answer: 0

Answer

Show hintCheckNextIdle

Quiz: successtrue statusfalse statusQuiz: \$?&& runs&& skips|| runs|| skipsQuiz: &&Quiz: ||exit 7test -ftest missing[*]Chain pitfallQuiz: pitfallls failset -etest -et&& with testFinal statusQuiz: meaning

Lab terminal

`$? = 0`set -o: off

Starter: try true; echo \$? · false && echo no · false || echo yes
lab1 true; echo \$?
0

lab\$ true · false · echo \$? · cmd && cmd · cmd || cmd

`true; echo $?``false; echo $?``true && echo ok``false && echo ok``false || echo fb``true || echo fb``A && B || C``test -f Makefile``set -e demo`

Chain evaluation

Which segments ran on the last `&&` / `||` line:
Run a line with `&&` or `||` to see steps.

Short-circuit rules

A && B

Run B only if A succeeds (status 0). If A fails, B is skipped.

A || B

Run B only if A fails (non-zero). If A succeeds, B is skipped.

Pitfall: `A && B || C` is not a clean if/else. If A succeeds and B fails, C still runs. Prefer `if/else` for real branches.

Cheat sheet

Item	Meaning
<code>\$?</code>	Exit status of the previous command (0 = success)
<code>true</code> / <code>false</code>	Status 0 / 1
<code>exit N</code>	Set status to N (lab: for the next <code>\$?</code>)
<code>test</code> / <code>[]</code>	File/string tests → 0 or 1
<code>set -e</code>	Abort the line after a failing simple command (lab demo)

• Success is 0; any non-zero is failure for `&&` / `||`.

• Lab files: `Makefile`, `rt1/top.v`, `notes.txt`.

Exit status lab

learn_unix — exit status lab

Real shell practice

```
wsl - module17-exit-status-lab/examples/exit_codes

# Track A - real Unix
# real shell session (Track A)

$ chmod u+x success.sh failure.sh check_file.sh

$ ./success.sh
Success script finished.
$ echo $?
0

$ ./failure.sh
Failure script: something went wrong.
$ echo $?
1

$ ./check_file.sh README.md
File exists: README.md
$ echo $?
0

$ ./check_file.sh missing.txt
File not found: missing.txt
$ echo $?
1

$ true && echo ok
ok

$ false || echo fallback
fallback
```

Real shell — exit codes

Real shell practice — try these

```
# chmod u+x ... — make the demo scripts executable
chmod u+x success.sh failure.sh check_file.sh
```

```
# ./success.sh — script exits 0 (success)
./success.sh
```

```
# echo $? — print exit status of the last command (expect 0)
echo $?
```

```
# ./failure.sh — script exits 1 (failure); status still printable
after
./failure.sh
```

```
echo $?
```

```
# ./check_file.sh README.md — exit 0 if the file exists
```

Pitfalls to watch

- Check dollar-question-mark immediately, any command, even echo, replaces it
- Do not treat ampersand-ampersand pipe-pipe as a full if-else
- And remember

Your turn

- Complete the checklist for at least one track, preferably both
- In the browser, finish a few challenges after the starter
- On the real shell
- When you are ready, take the short quiz, then continue to safe scripting